The Complete Beginner's Guide to AI-Powered Development

From Installing VS Code to Running Autonomous AI Agents on EC2

By Marceau Solutions *Last Updated: March 2026*

Table of Contents

1. What This Guide Covers
2. The Big Picture
3. Phase 1: Your Local Machine Setup
4. Phase 2: Claude Code — Your AI Pair Programmer
5. Phase 3: Building Your First AI-Powered Project
6. Phase 4: Amazon EC2 — Your Cloud Server
7. Phase 5: Deploying AI Agents to EC2
8. Phase 6: The Three-Agent Architecture
9. Phase 7: Integration — Making It All Work Together
10. Glossary

1. What This Guide Covers

This guide takes you from zero to running a full AI-powered development operation. By the end, you will have:

- **VS Code** installed and configured as your development environment
- **Claude Code** integrated into VS Code as your AI pair programmer
- An **Amazon EC2 server** running in the cloud 24/7
- **Three AI agents** working for you: one on your laptop, two on your server
- All of them talking to each other and doing real work

No prior programming experience is assumed. Every step is explained.

---

2. The Big Picture

Here is what the full system looks like when it is running:

```
YOUR LAPTOP (Mac/Windows)
  VS Code + Claude Code Extension
    = "Claude Code" — your primary AI agent
    = You talk to it, it writes code, reads files, runs commands
    = This is your command center

AMAZON EC2 (Cloud Server — runs 24/7)
  Clawdbot / OpenClaw — Telegram-connected AI agent
    = Responds to messages on Telegram
    = Runs automated tasks on a schedule
    = Monitors your systems while you sleep

  Ralph — Autonomous execution agent
    = Reads PRD (Product Requirement Documents) and builds things
    = Runs without human input
    = Handles background processing

n8n (Workflow Automation — runs on EC2)
  = Visual workflow builder (like Zapier but self-hosted)
  = Connects your agents to email, SMS, calendars, databases
  = The "nervous system" that ties everything together
```

You start at the top (VS Code) and build your way down. Each phase builds on the last.

---

3. Phase 1: Your Local Machine Setup

3.1 Install VS Code

What it is: Visual Studio Code is a free code editor made by Microsoft. Think of it as Microsoft Word, but for code. It is the industry standard.

Download: Go to https://code.visualstudio.com and click the big download button. It auto-detects your operating system (Mac, Windows, or Linux).

Install:

- **Mac:** Open the downloaded `.zip` file. Drag "Visual Studio Code" into your Applications folder. Open it from Applications.

- **Windows:** Run the downloaded `.exe` installer. Check "Add to PATH" when prompted. Click through the installer.

First launch: Open VS Code. You will see a Welcome tab. Close it. You now have a professional code editor.

3.2 Install a Terminal

VS Code has a built-in terminal. Open it with:

- **Mac:** Press `Ctrl + backtick` (the key above Tab)
- **Windows:** Same shortcut, or go to View > Terminal

This terminal is where you will type commands. It is a direct line to your computer's operating system.

3.3 Install Python (if you do not have it)

Most AI tools are built in Python. Check if you have it:

```
python3 --version
```

If you see a version number (like `Python 3.11.5`), you are good. If not:

- **Mac:** `brew install python3` (install Homebrew first from https://brew.sh if needed)
- **Windows:** Download from https://python.org. Check "Add Python to PATH" during install.

3.4 Install Node.js

Some tools (including Claude Code CLI) need Node.js:

```
node --version
```

If not installed:

- **Mac:** `brew install node`
- **Windows:** Download from https://nodejs.org (LTS version)

3.5 Install Git

Git is version control — it tracks every change you make so you can undo anything.

```
git --version
```

If not installed:

- **Mac:** It will prompt you to install Xcode Command Line Tools. Say yes.
- **Windows:** Download from https://git-scm.com

Configure Git (replace with your info):

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

3.6 Create Your Project Folder

```
mkdir ~/dev-sandbox
cd ~/dev-sandbox
git init
```

You now have a version-controlled project folder. Everything you build goes here.

---

4. Phase 2: Claude Code — Your AI Pair Programmer

4.1 What Is Claude Code?

Claude Code is an AI agent made by Anthropic that lives inside your terminal and VS Code. It can:

- Read and write files on your computer
- Run terminal commands
- Search your entire codebase
- Write, debug, and refactor code
- Create git commits and pull requests
- Browse the web for documentation
- Plan and execute multi-step tasks

It is not a chatbot that gives you suggestions. It is an agent that does the work.

4.2 Get an Anthropic Account

1. Go to https://console.anthropic.com
2. Create an account
3. Add a payment method (Claude Code uses the API, which is pay-per-use)
4. Go to API Keys and create a new key
5. Copy the key — you will need it in a moment

Typical cost: $5-20/month for active development use.

4.3 Install Claude Code CLI

In your terminal:

```
npm install -g @anthropic-ai/claude-code
```

Set your API key:

```
export ANTHROPIC_API_KEY="sk-ant-your-key-here"
```

To make this permanent, add that line to your shell profile:

- **Mac (zsh):** `echo 'export ANTHROPIC_API_KEY="sk-ant-your-key-here"' >> ~/.zshrc`
- **Windows (PowerShell):** Add to your PowerShell profile

4.4 Install the VS Code Extension

1. Open VS Code
2. Click the Extensions icon in the left sidebar (or press `Cmd+Shift+X` on Mac / `Ctrl+Shift+X` on Windows)
3. Search for "Claude Code"
4. Install the official Anthropic extension
5. It will detect your API key automatically

4.5 Your First Conversation

Open the Claude Code panel in VS Code (look for it in the sidebar or bottom panel). Type:

```
Create a Python script that prints "Hello, AI World" and the current date
```

Watch what happens. Claude Code will:

1. Create a new file

2. Write the code
3. Ask if you want to run it
4. Show you the output

You just used an AI agent to write and execute code. Everything from here builds on this.

4.6 Key Claude Code Concepts

Context: Claude Code reads your project files to understand your codebase. The more organized your project, the better it works.

Tools: Claude Code has tools — it can read files, write files, run bash commands, search code, and more. When you ask it to do something, it picks the right tool.

CLAUDE.md: Create a file called `CLAUDE.md` in your project root. This is your instruction manual for Claude Code. Write what your project does, what rules to follow, what to never do. Claude reads this at the start of every conversation.

Approval: By default, Claude Code asks permission before running commands or editing files. You can adjust this in settings.

---

5. Phase 3: Building Your First AI-Powered Project

5.1 Project Structure

Ask Claude Code to set up a project:

```
Create a basic Python project structure with:
- src/ folder for code
- requirements.txt for dependencies
- .env for environment variables (add to .gitignore)
- README.md
- CLAUDE.md with basic project instructions
```

5.2 Environment Variables

Never put API keys or passwords directly in your code. Use a `.env` file:

```
# .env (NEVER commit this to git)
```

```
ANTHROPIC_API_KEY=sk-ant-your-key
TWILIO_SID=your-twilio-sid
DATABASE_URL=sqlite:///data/app.db
```

Load them in Python:

```
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("ANTHROPIC_API_KEY")
```

5.3 Your First Real Script

Ask Claude Code:

```
Build a script that:
1. Reads a CSV file of business leads
2. Scores them 1-100 based on industry and location
3. Outputs a sorted list of the top 10
Use pandas. Put it in src/lead_scorer.py
```

This is the pattern: you describe what you want in plain English, Claude Code builds it, you review and approve.

5.4 Git Workflow

After Claude Code creates or modifies files:

```
Commit these changes with a descriptive message
```

Claude Code will stage the files, write a commit message, and create the commit. Your work is now version-controlled and recoverable.

---

6. Phase 4: Amazon EC2 — Your Cloud Server

6.1 What Is EC2?

Amazon EC2 (Elastic Compute Cloud) is a virtual server that runs 24/7 in Amazon's data centers. Think of it as a

computer in the cloud that never sleeps. You connect to it remotely from your laptop.

Why you need one:

- Your laptop turns off. EC2 does not.
- AI agents that monitor email, respond to messages, run scheduled tasks — they need a machine that is always on.
- It has a public IP address, so external services (webhooks, APIs) can reach it.

6.2 Create an AWS Account

1. Go to https://aws.amazon.com
2. Click "Create an AWS Account"
3. Enter your email, create a password
4. Add a payment method (you get 12 months of free tier — a small EC2 instance is free)
5. Choose the "Basic (Free)" support plan

6.3 Launch Your First EC2 Instance

1. Go to the EC2 Dashboard (search "EC2" in the AWS console)
2. Click "Launch Instance"
3. Configure:
   - **Name:** `my-ai-server`
   - **AMI:** Amazon Linux 2023 (free tier eligible)
   - **Instance type:** `t2.micro` (free tier) or `t3.small` ($15/month, recommended for real work)
   - **Key pair:** Click "Create new key pair", name it `my-ec2-key`, download the `.pem` file. **Save this file. You cannot download it again.**
   - **Security group:** Allow SSH (port 22) from your IP, HTTP (port 80), HTTPS (port 443)
4. Click "Launch Instance"

6.4 Connect to Your Server

Move your key file somewhere safe:

```
mkdir -p ~/.ssh
mv ~/Downloads/my-ec2-key.pem ~/.ssh/
chmod 400 ~/.ssh/my-ec2-key.pem
```

Find your instance's public IP in the EC2 dashboard. Connect:

```
ssh -i ~/.ssh/my-ec2-key.pem ec2-user@YOUR_PUBLIC_IP
```

You are now logged into your cloud server. Everything you type runs on that machine, not your laptop.

6.5 Set Up the Server

Run these commands on EC2:

```
# Update the system
sudo yum update -y

# Install Python 3.11
sudo yum install python3.11 python3.11-pip -y

# Install Node.js (for n8n and Claude Code)
curl -fsSL https://rpm.nodesource.com/setup_20.x | sudo bash -
sudo yum install nodejs -y

# Install Git
sudo yum install git -y

# Create your workspace
mkdir -p ~/workspace
cd ~/workspace
git init
```

6.6 Security Basics

Never do these things:

- Never put your `.pem` key file in a git repository
- Never open all ports in your security group (only open what you need)
- Never run everything as root
- Never put API keys in code files — always use `.env`

Always do these things:

- Keep your server updated (`sudo yum update -y` weekly)
- Use SSH keys, not passwords
- Back up your data regularly
- Monitor your AWS bill

---

7. Phase 5: Deploying AI Agents to EC2

7.1 Install Claude Code on EC2

SSH into your server and install:

```
npm install -g @anthropic-ai/claude-code
```

Set your API key in the environment:

```
echo 'export ANTHROPIC_API_KEY="sk-ant-your-key"' >> ~/.bashrc
source ~/.bashrc
```

Now you can run Claude Code on your server, just like on your laptop.

7.2 Install n8n (Workflow Automation)

n8n is the automation layer. It connects your AI agents to email, SMS, webhooks, databases, and hundreds of other services.

```
# Install n8n globally
npm install -g n8n

# Create data directory
mkdir -p ~/.n8n

# Start n8n (basic)
n8n start
```

For production, run n8n as a service with PM2:

```
npm install -g pm2
pm2 start n8n -- start
pm2 save
pm2 startup  # auto-start on reboot
```

n8n runs on port 5678 by default. Access it at `http://YOUR_EC2_IP:5678`.

For a custom domain (like `n8n.yourdomain.com`), set up Nginx as a reverse proxy with SSL via Let's Encrypt.

7.3 Set Up Clawdbot / OpenClaw

Clawdbot (also called OpenClaw) is a Telegram-connected AI agent. It:

- Receives messages from you on Telegram
- Processes them using Claude's API

- Can run commands, check your systems, send notifications
- Runs 24/7 on EC2

Step 1: Create a Telegram Bot

1. Open Telegram and message `@BotFather`
2. Send `/newbot`
3. Name it (e.g., "My AI Assistant")
4. Choose a username (e.g., `my_ai_assistant_bot`)
5. BotFather gives you a token like `7123456789:AAHxxx...` — save this

Step 2: Get Your Chat ID

1. Message your new bot on Telegram (say "hello")
2. Visit: `https://api.telegram.org/botYOUR_TOKEN/getUpdates`
3. Find `"chat":{"id":YOUR_CHAT_ID}` in the response
4. Save this number

Step 3: Deploy

The core of Clawdbot is a Node.js application that:

- Listens for Telegram messages via the Bot API
- Passes them to Claude's API for processing
- Returns Claude's response to Telegram
- Has access to tools (file system, commands, web search)

```
# On EC2
mkdir ~/clawdbot && cd ~/clawdbot
npm init -y
npm install node-telegram-bot-api @anthropic-ai/sdk dotenv
```

Create `.env`:

```
TELEGRAM_BOT_TOKEN=your-bot-token
TELEGRAM_CHAT_ID=your-chat-id
ANTHROPIC_API_KEY=sk-ant-your-key
```

The bot application code connects these pieces: incoming Telegram message becomes a Claude API call, Claude's response gets sent back to Telegram. Claude can also be given tools (read files, run commands, check databases) that let it take actions on your server.

Run with PM2:

```
pm2 start app.js --name clawdbot
pm2 save
```

Now message your bot on Telegram. It responds using Claude.

7.4 Set Up Ralph (Autonomous Agent)

Ralph is a headless agent — no chat interface. It reads task files (PRDs — Product Requirement Documents) and executes them autonomously.

How Ralph works:

1. You (or another agent) writes a PRD file describing what needs to be built
2. Ralph picks it up from a watched directory
3. Ralph uses Claude Code to plan and execute the work
4. Ralph writes results and logs what it did
5. Ralph moves to the next PRD

Setup:

```
mkdir -p ~/ralph/inbox ~/ralph/completed ~/ralph/logs
```

Ralph is essentially a Claude Code session running in a loop:

```
# Ralph's execution loop (simplified concept)
while true; do
  PRD=$(ls ~/ralph/inbox/*.md | head -1)
  if [ -n "$PRD" ]; then
    claude --print "Read $PRD and execute all tasks described in it" \
      --allowedTools Edit,Write,Bash,Read,Glob,Grep
    mv "$PRD" ~/ralph/completed/
  fi
  sleep 60
done
```

In practice, Ralph uses the Claude Agent SDK for more sophisticated execution with error handling, progress tracking, and result reporting.

---

8. Phase 6: The Three-Agent Architecture

8.1 Why Three Agents?

Each agent has a different strength:

Agent	Location	Interface	Strength
Claude Code	Your laptop (VS Code)	VS Code panel / terminal	Interactive development, complex reasoning, your direct collaborator
Clawdbot/ OpenClaw	EC2	Telegram	24/7 monitoring, quick tasks from your phone, notifications
Ralph	EC2	Headless (PRD-driven)	Autonomous execution, background builds, batch processing

8.2 How They Communicate

The agents do not talk to each other directly. They communicate through shared state:

Pipeline Database (SQLite): All agents read and write to the same database. When Claude Code creates a lead, Clawdbot can see it. When Ralph processes a task, the results appear in the database.

Shared File System: On EC2, Clawdbot and Ralph share the same file system. A file written by one can be read by the other. Your laptop syncs via git push/pull or SCP.

Handoff Files: A `HANDOFF.md` file tracks what needs to be passed between agents:

```
# HANDOFF.md
## For Clawdbot
- [ ] Monitor inbox for replies to Tuesday's outreach

## For Ralph
- [ ] Build PDF report from this week's pipeline data

## For Claude Code (next session)
- [ ] Review Ralph's output and refine
```

n8n Webhooks: n8n can trigger any agent via HTTP webhooks. Example: an incoming email triggers n8n, which writes a task for Clawdbot, which processes it and notifies you on Telegram.

8.3 The Daily Flow

Here is how a typical day works with all three agents:

6:30 AM — Clawdbot sends you a Telegram digest: pipeline status, emails, calendar, system health.

7:00 AM — You review on your phone. Reply "yes schedule" to approve the day's time blocks.

9:00 AM — You open VS Code. Claude Code reads your project, checks the handoff file, and picks up where yesterday left off.

Throughout the day — Claude Code builds features, fixes bugs, runs tests. You review and approve.

While you work — Clawdbot monitors for incoming emails, SMS replies, webhook events. Sends you Telegram alerts for anything urgent.

Background — Ralph processes any PRDs in its inbox. Builds reports, generates content, runs batch operations.

10:00 PM — You close your laptop. Clawdbot and Ralph keep working on EC2. They will be there when you wake up.

---

9. Phase 7: Integration — Making It All Work Together

9.1 Sync Your Code Between Laptop and EC2

Option 1: Git (recommended)

```
# On your laptop — push changes
git add . && git commit -m "update" && git push

# On EC2 — pull changes
cd ~/dev-sandbox && git pull
```

Option 2: SCP (direct file copy)

```
# Copy a file from laptop to EC2
scp -i ~/.ssh/my-ec2-key.pem ./file.py ec2-user@YOUR_IP:~/workspace/

# Copy from EC2 to laptop
scp -i ~/.ssh/my-ec2-key.pem ec2-user@YOUR_IP:~/workspace/file.py ./
```

9.2 Environment Variable Sync

Both your laptop and EC2 need the same API keys. Maintain a `.env` file on each:

```
# Sync .env from laptop to EC2
scp -i ~/.ssh/my-ec2-key.pem ./.env ec2-user@YOUR_IP:~/workspace/.env
```

Critical rule: Never commit `.env` to git. Add it to `.gitignore`.

9.3 Database Sync

If using SQLite, sync the database file:

```
# Push local DB to EC2
scp -i ~/.ssh/my-ec2-key.pem ./data/pipeline.db ec2-user@YOUR_IP:~/data/pipeline.db

# Pull EC2 DB to local
scp -i ~/.ssh/my-ec2-key.pem ec2-user@YOUR_IP:~/data/pipeline.db ./data/pipeline.db
```

For production, consider PostgreSQL on EC2 so all agents connect to one database without file syncing.

9.4 n8n as the Nervous System

n8n connects everything with visual workflows:

Example workflow: Inbound Lead Processing

1. Trigger: Webhook receives form submission
2. Code node: Score the lead (1-100)
3. If score > 70: Add to pipeline database
4. HTTP Request: Notify Clawdbot via Telegram
5. If score > 90: Send SMS alert to your phone
6. Log everything to a Google Sheet

Example workflow: Morning Digest

1. Trigger: Cron schedule (6:30 AM daily)
2. Code node: Query pipeline database for stats
3. HTTP Request: Check Gmail for unread
4. HTTP Request: Get today's calendar events
5. Code node: Format everything into a message
6. Telegram node: Send digest to your chat

9.5 Custom Domain Setup (Optional but Professional)

1. Buy a domain on Namecheap or similar
2. Point DNS A record to your EC2 IP
3. Install Nginx on EC2:

```
sudo yum install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
```

Python	Programming language used for most AI and automation tools
Ralph	Autonomous AI agent that executes PRDs without human input
SCP	Secure Copy — transfers files between your laptop and EC2
SQLite	A lightweight database stored as a single file
SSH	Secure Shell — encrypted connection to a remote server
VS Code	Visual Studio Code — the code editor
Webhook	A URL that receives data when an event happens

What To Do Next

1. **Start with Phase 1-2.** Get VS Code and Claude Code working. Spend a few days just building things with Claude Code on your laptop.
2. **Move to Phase 4** when you are ready for 24/7 operations. Launch an EC2 instance and get comfortable with SSH.
3. **Deploy one agent at a time.** Start with Clawdbot (Telegram is the easiest interface). Add Ralph later when you have tasks that need autonomous execution.
4. **Build incrementally.** Every complex system was once a simple script. Start small, add capabilities as you need them.
5. **Use Claude Code to build everything.** The irony of this guide is that the best way to follow it is to paste each section into Claude Code and say "help me do this."

This guide was built using the exact tools it describes. The system works. Now go build yours.

Marceau Solutions | marceausolutions.com